

27th-09-2026

International Cybersecurity Of Digital Forensic Academy (ICDFA)

PROGRAMME: Advance Critical Infrastructure Security (ACIS)

Prepared by: Aminu Ibrahim Jalo
Course: ACICP301-INTRO TO CRITICAL INFRASTRUCTURE
FOR IT LEARNERS

Title: PLC PROGRAMMING LANGUAGES

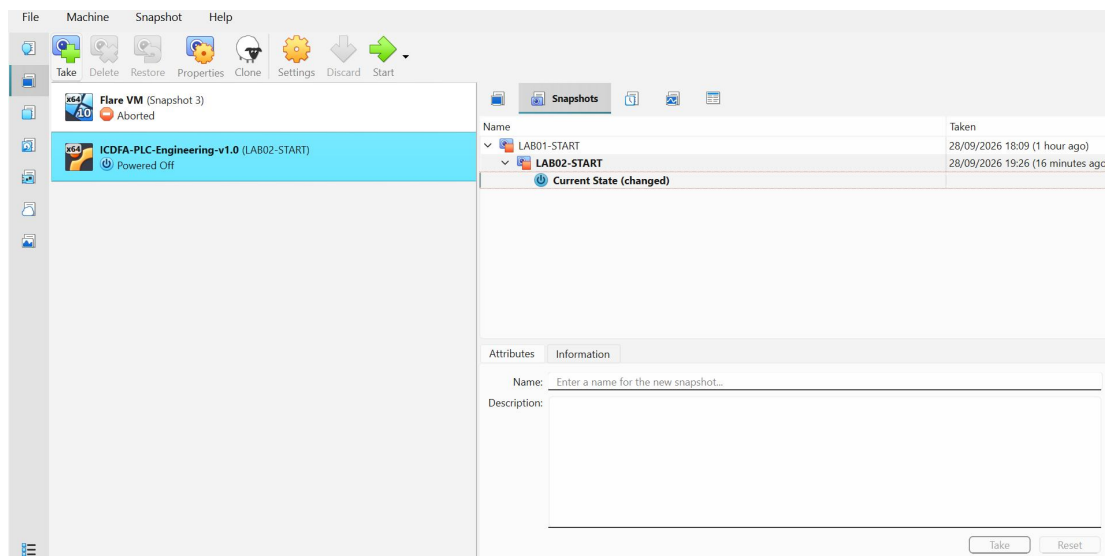
Assignment: Week 1-Lab 2 Report
C11/26/ACIS/17359

Instructor : Aminu Idris

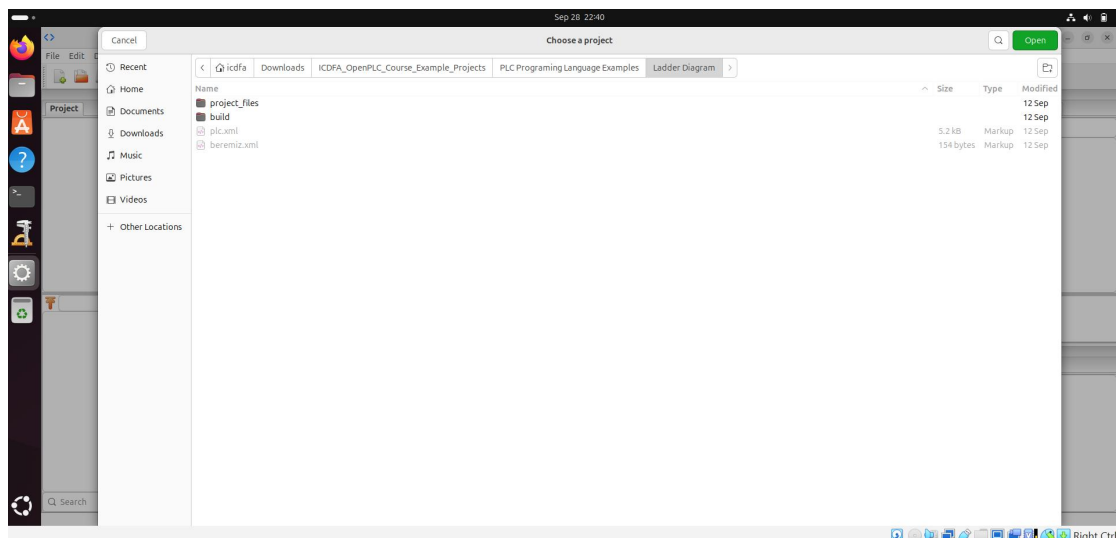
Student Name	Aminu Ibrahim Jalo
Registration Number	C11/26/ACIS/17359
Signature	
Date	27 th -09-2026

Procedure

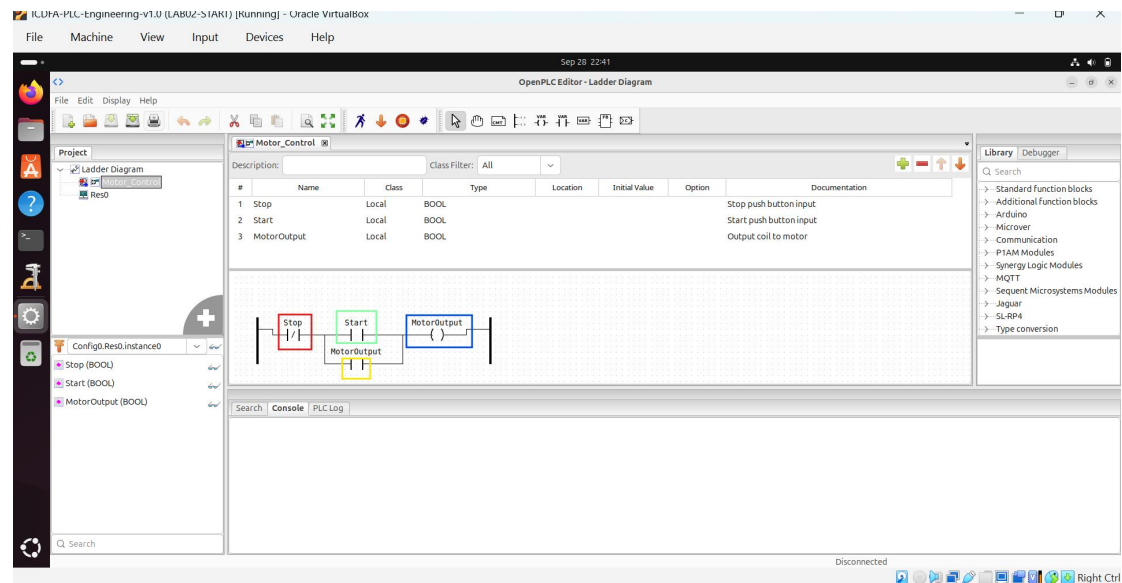
Step 1. Start from the clean IC DFA PLC VM snapshot and sign in to the PLC training environment.



Step 2. Launch OpenPLC Editor and open the Ladder Diagram example from the PLC Programming Language Examples directory.



Step 3. Open Res0 > instance0 (Motor_Control). Identify the normally open Start contact, normally closed Stop contact, MotorOutput latch contact and MotorOutput coil.



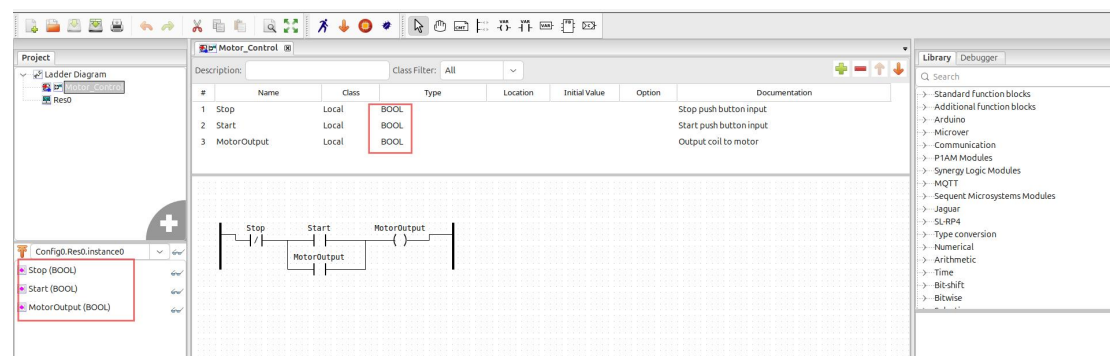
RED- Closed Stop contact

GREEN- Open Start contact

YELLOW- MotorOutput latch contact

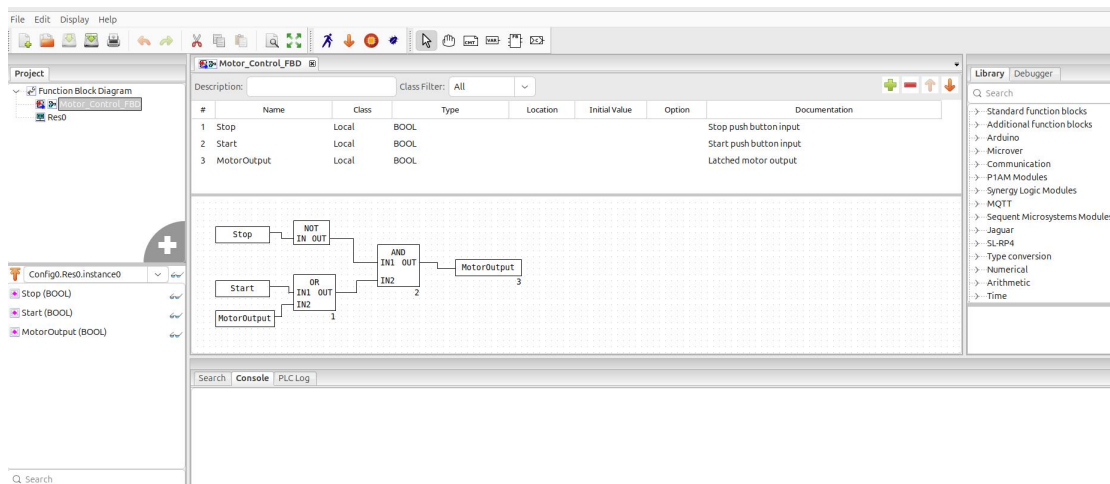
BLUE- MotorOutput coil

Step 4. Open the tag/variable list. Confirm that the motor-control tags use Boolean values and record the purpose of each tag in your worksheet.



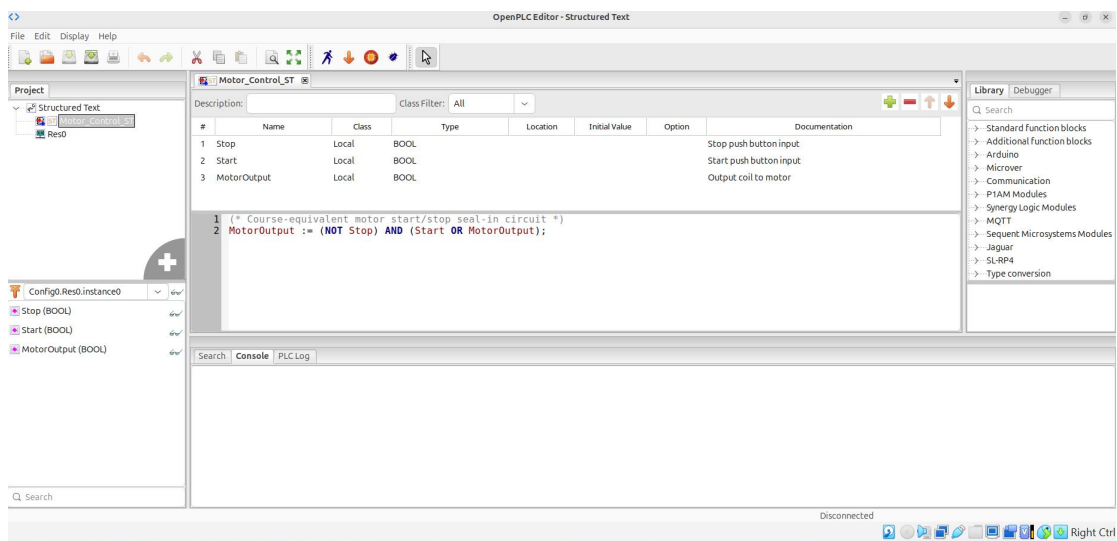
All are Boolean because each is a two-state (discrete) signal.

Step 5. Return to File > Open and load the Function Block Diagram example. Open Motor_Control_FBD and identify the block that maintains the motor output until a stop condition occurs.



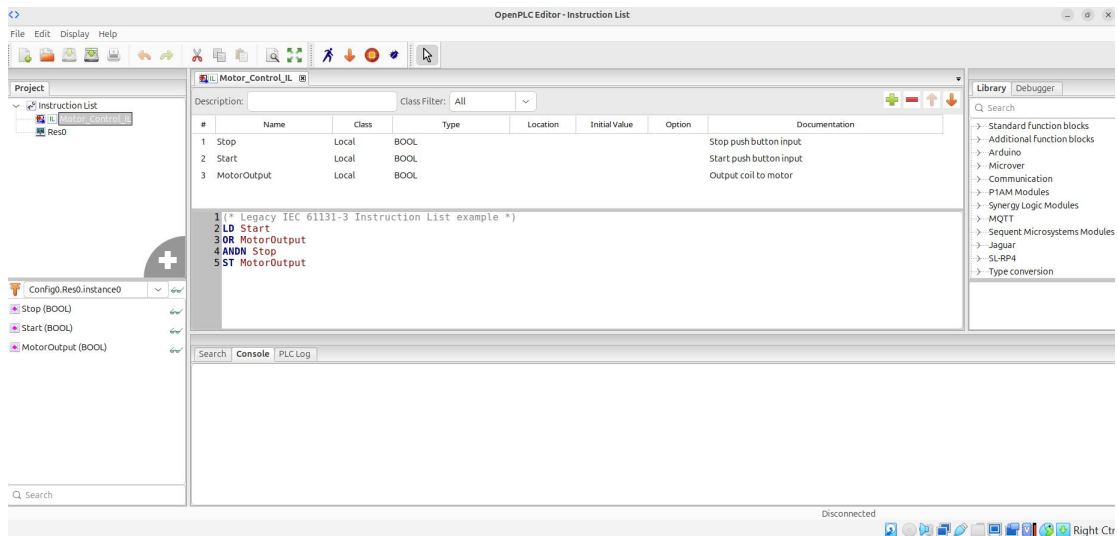
The block that maintains the output is the **SR or RS (set/reset) flip-flop**. Start feeds the Set input and Stop feeds the Reset input. The output stays TRUE after Start is released until Stop resets it.

Step 6. Open the Structured Text example and inspect MotorControlST. Describe how the same start/stop behavior is represented using text rather than graphical contacts and coils.



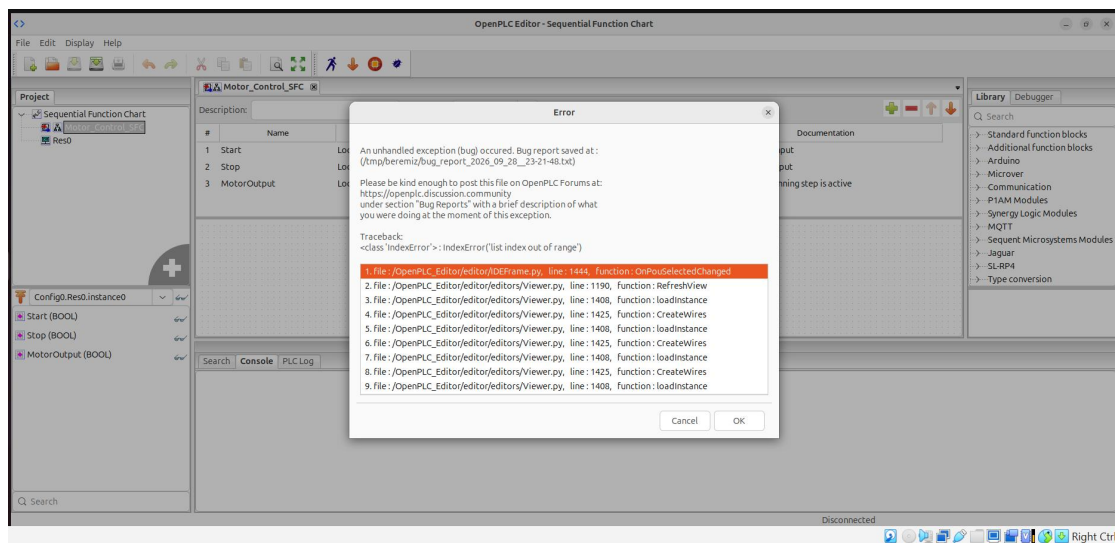
OR MotorOutput is the latch, and AND NOT Stop is the stop condition.

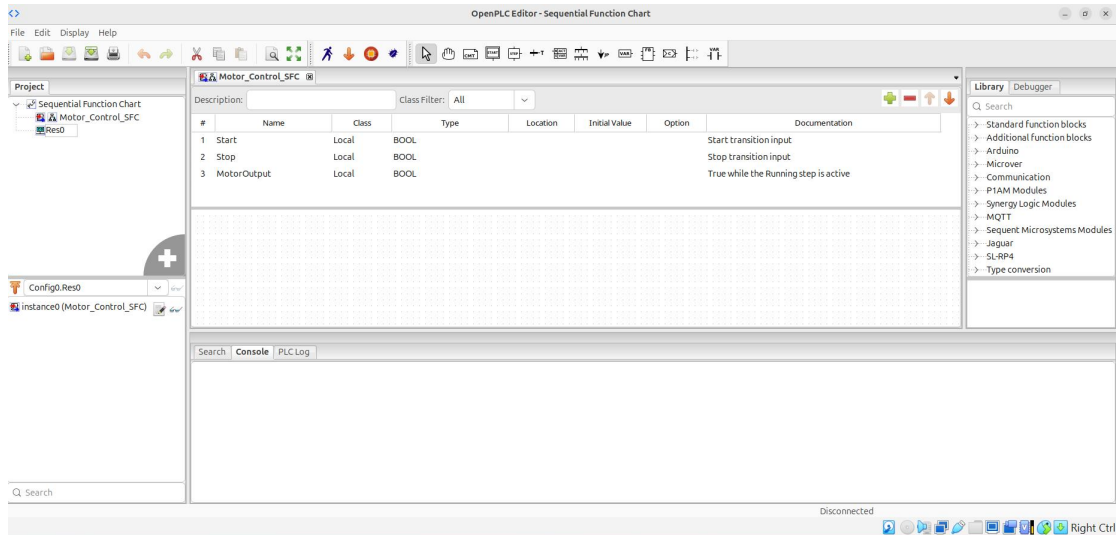
Step 7. Open the Instruction List example and inspect MotorControl. Record the main visual difference between this low-level textual representation and Structured Text.



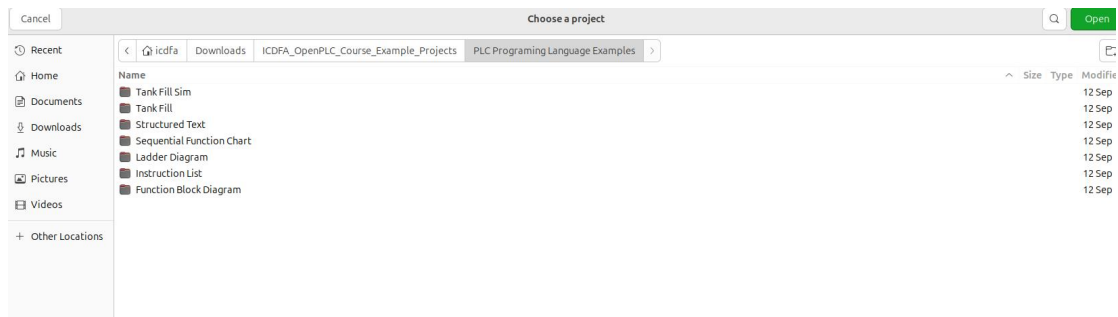
The main difference is that IL is a line-by-line, assembly-like sequence of one operation per line using an accumulator, while ST uses a single high-level expression with operators and structured syntax.

Step 8. Open the Sequential Function Chart example and inspect MotorControl_SFC. Identify how steps/transitions express process sequence.





Step 9. Complete the language comparison matrix in the lab report for all five representations.



Representation	Strengths	Limitations	Industrial use case
Ladder Diagram (LD)	Familiar to electricians, easy to trace and troubleshoot	Gets bulky for math and complex logic	Motor start/stop, interlocks
Function Block Diagram (FBD)	Reusable blocks, clear signal flow	Large diagrams become crowded	Process control loops, timers, PID
Structured Text (ST)	Compact, good for calculations and conditions	Harder for non-programmers to read	Data handling, recipes, calculations
Instruction List (IL)	Very compact, close to machine level	Hard to read and maintain, largely deprecated	Small legacy programs
Sequential Function Chart (SFC)	Clear step-by-step sequencing	Needs other languages inside steps and actions	Batch processes, machine cycles

Step 10.

Relection:

All five express one control objective: the motor starts when Start is pressed, keeps running after Start is released, and stops when Stop is pressed. Ladder does it with a latch contact, FBD with a set/reset block, ST with one Boolean expression, IL with a short instruction sequence, and SFC with states and transitions. The tags and the behavior are identical, and only the notation changes.

Knowledge Check

1. Why is Ladder Diagram intuitive for technicians familiar with relay logic?

Response: It mimics relay wiring diagrams, with power rails, contacts and coils, so it reads like the circuits they already know.

2. What is one advantage of Structured Text for complex calculations or conditions?

Response: It handles calculations, comparisons and complex conditions compactly, where ladder would need many rungs.

3. When might a Sequential Function Chart be more appropriate than a single ladder rung?

Response: When the process runs in ordered stages (for example fill, heat, mix, drain), because steps and transitions show the sequence and current state clearly.

4. Why must the meaning of tags remain consistent when the representation/language changes?

Response: The same physical signals connect to the program in every language. If Stop meant something different in another representation, the programs would behave differently or unsafely, and troubleshooting and maintenance would become unreliable.